

高确定性商业系统的 Agent 落地实践

2026.7 深圳

运满满·找货AI：业务背景

AGENTIC AI
超级智能系统架构峰会



中国最大数字货运平台，连接货主与卡车司机；找货AI 帮司机在海量货源中高效找到合适的货

平台规模

6340万

履约订单（2025 Q3）

335万

发货方月活 MAU（平均）

448万

活跃卡车司机（近12个月）

33.6亿

营收 RMB（2025 Q3）

为什么这门生意适合 Agent

客单价高

单笔运费金额可观
值得为每一单
投入智能决策

交易非标

货源千差万别
司机服务也非标
规则难以穷举，需理解决策

居间撮合提效

居间撮合
可显著提升货与人的
匹配效率

全流程线上化

交易全程线上完成
数据闭环完整
决策可被反馈持续优化

数据来源：Full Truck Alliance（满帮集团）2025 年第三季度业绩公告

目录

CONTENT

- 01 核心约束与实践方向
- 02 事件驱动与执行引擎
- 03 意图编译与快速执行
- 04 分层授权与能力治理

01

核心约束与实践方向

核心约束与实践方向

AGENTIC AI
超级智能体系统架构峰会



约束一：感知必须确定且成本可控

业务确定性

自主感知无法确定"什么变了"
百万司机×持续Loop = 成本扛不住

关键性事件必须被确定性告知



实践一：事件驱动，按需唤醒

Agent 不轮询，由业务事件确定性唤醒
不轮询 = 不花token + 感知完全确定

约束二：推货必须快且成本可控

秒级推送延迟

好货成交窗口分钟级
逐票判断 = 推货慢 + 成本扛不住

Agent运行几十秒--好货早没了



实践二：意图编译为执行脚本

理解一次，编译一次，规则引擎秒级匹配
LLM负责理解，确定性引擎负责执行

约束三：动作风险必须可控

业务风险可控

不同动作风险等级完全不同
推错损伤信任，成交错单损失订金

取消订单 → 扣订金



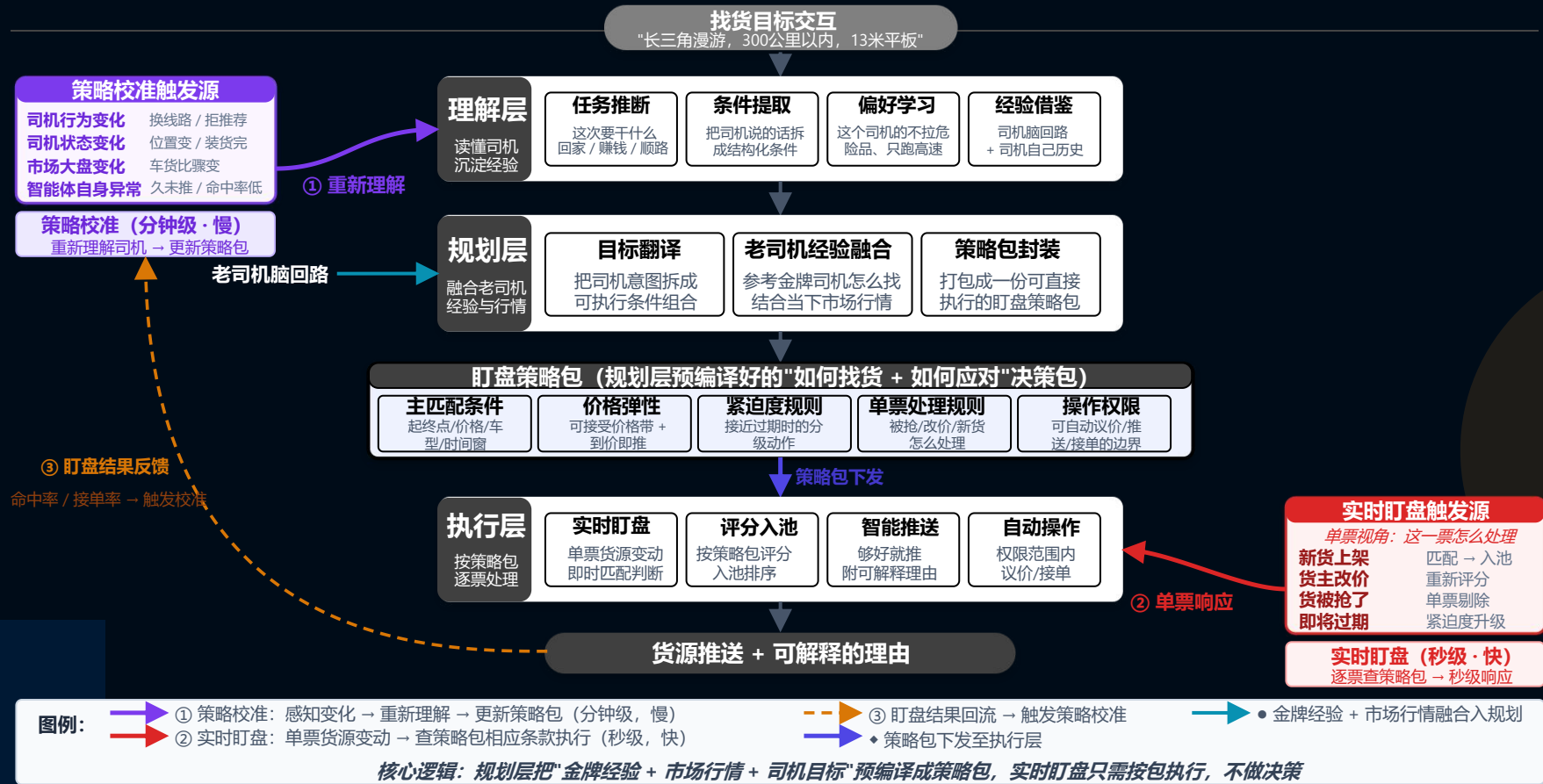
实践三：动作分层授权

不同动作不同信任层级
按风险分级，出错可审计、可回溯

运满满·找货AI：运行流程

AGENTIC AI
超级智能体系统架构峰会

DataFun.



02

事件驱动与执行引擎

实践一：事件驱动，按需唤醒

问题

Agent 不能 7x24 持续运行。百万司机 × 24h × 持续 **Agent Loop** = token 成本天文数字。
但司机的找货任务是长程的（开启到结束至少持续数小时）——如何在非持续运行的前提下保持长程状态？

方案：自定义事件类型 × Event2NL × 调度策略

事件类型	触发时机	调度策略	说明
conversation	司机发消息	preempt （立即抢占）	打断正在运行的 tick
tick	周期性触发	coalesce （合并丢弃）	堆积多个只保留最新
cold_start	司机首次上线	queue （排队等待）	先到先服务
intent_report	意图变更上报	queue	触发策略重编译
deal	成交事件	queue	更新司机找活目标

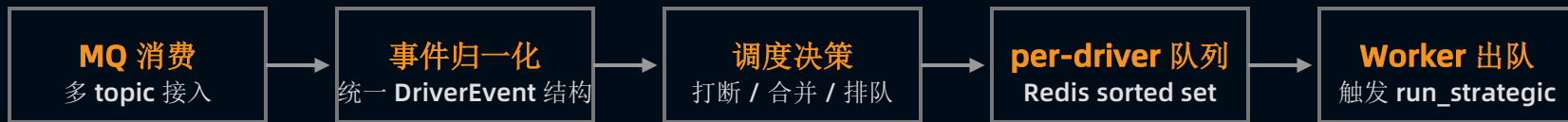
核心观点

Agent 本质仍是长程的（有状态、有记忆、有持续目标），但它不"一直跑"——只在有事时被唤醒跑一次。
好处：(1) 不轮询就不花 **token** (2) 感知更可控——确定性地告诉 **Agent**"什么变了"

参考：Anthropic Agent Loop – evaluate → tool call → result → repeat（我们的改造：repeat 由事件触发而非 **Agent** 自主循环）

工程实现：事件调度引擎

AGENTIC AI
超级智能体系系统架构峰会



三个核心机制

打断抢占 (Preempt)

司机发消息 → 写 **cancel flag**
→ 正在执行的 **Worker** 检测到 **flag**
→ 中止当前 **run**
→ 新事件入队优先执行
latency: < 500ms (cancel检测间隔)

coalesce 合并

多个 **tick** 事件堆积时
→ 只保留最新一条
→ 丢弃中间过程
避免无意义重复执行
效果: tick风暴不会压垮系统

分布式串行保证

同一司机同时只有一个 **Worker** 执行
→ **pod** 归属 + **Redis** 锁
→ 锁续期 + 超时自动释放
→ 保证状态一致性
无竞态: 同driver串行执行

实现结果

事件驱动 → 统一调度入口 → 每个司机串行执行、互不干扰

Agent 每次被唤醒, 已经知道"为什么醒来"和"上次停在哪"--**event** 提供 **why**, **Redis state** 提供 **where**

03

意图编译与快速执行

实践二：意图编译为执行脚本

问题

好货成交窗口只有几分钟。**Agent loop**运行几十秒且时间不可控。逐票让 **Agent** 判断“这票货适不适合”——永远抢不到。但司机的找货意图是自然语言表达的（“我想跑上海到杭州冷链 价格别太低”）——如何用确定性规则秒级执行模糊意图？

方案：两阶段分离

Slow Loop（慢循环）

Agent + LLM 理解找货意图



编译为结构化规则脚本（**WatchPlan**）



写入 **Redis**，等待执行

时延容忍：分钟级

产出
→
WatchPlan Script

Fast Loop（快循环）

规则引擎拿着 **WatchPlan** 执行



每票新货源 → 规则匹配 → 评级 → 推送决策



不依赖 **Agent**

时延要求：秒级

核心观点

Agent 的核心价值在于“理解司机找货意图并把决策逻辑编译为可执行规则”。

LLM 负责理解——理解完了，就把接力棒交给确定性引擎。手脑分离。

类比具身智能：目标 → 大脑 → 神经 → 肌肉

参考：Anthropic Dynamic Workflows – “moving the plan into code”（我们的做法：把意图编译为规则脚本，执行不依赖 **Agent**）

WatchPlan: 约束 → 规则化编译产物

AGENTIC AI
超级智能体系统架构峰会



输入（司机自然语言）

"我想跑附近的冷链货，价格别太低，最好 30 公里以内能装货"

编译产出（结构化 WatchPlan）

召回条件（40+ 维度）

出发地：上海
目的地：杭州
货物类型：冷链
装货距离： $\leq 30\text{km}$
价格下限：市场均价 $\times 0.8$
车型匹配：冷藏车
时间窗：当前可用时段
...共 40+ 维度，由 LLM 根据对话推断填充
作用：决定"看哪些货"

过滤规则（阻断条件）

C1: 黑名单货主 → 直接拦截
C2: 超重超限 → 拦截
C3: 已推过未反馈 → 降权
C4: 同线路饱和 → 限流
C5: 司机已接单 → 暂停

每条规则独立判定
命中即阻断，无需模型介入

作用：决定"哪些不能推"

执行脚本（策略参数）

评级矩阵：
完全匹配 → 自动推送
高度匹配 → 入池竞争
部分匹配 → 入池低优
不匹配 → 丢弃

推送策略：TopN=3，间隔 $\geq 5\text{min}$
门控参数：日推上限=20

作用：决定"怎么推、推多少"

WatchPlan 是 Agent 每次"思考"的输出产物。只要找货策略不变，WatchPlan 不需要重新生成——规则引擎可以持续执行。

Fast Loop: 规则引擎秒级执行

AGENTIC AI
超级智能体系架构峰会



执行链路



执行特征



反馈闭环: Fast Loop → tick → Strategic Loop



快慢 Loop 通过 tick 事件形成正向闭环--Fast Loop 负责执行, Strategic Loop 负责纠偏。

04

分层授权与能力治理

实践三： 分层授权体系

问题

Agent 能做的事越来越多——推货、打电话、帮加价、帮抢单。这些动作的风险等级完全不同。
如果所有动作都用同一套执行策略，要么过于保守（什么都不敢做），要么过于激进（出了错收不回来）。

方案：5 级授权体系

层级	名称	动作举例	执行策略	回退机制
L0	观察	记录行为、分析偏好	只读，不产生外部动作	无需回退
L1	建议推送	货源入池 → 门控 → 推送	系统自动执行，门控拦截即停止	门控拦截即停止
L2	直接推送	A1 评级货源跳过池竞争直推	跳过池竞争，直接触达司机	频次超限 → 降级为 L1
L3	主动外呼	AI 电话通知司机优质货源	需司机预授权 + 时段限制 + 日次上限	司机投诉 → 关闭外呼
L4	代操作	帮司机加价、帮司机抢单	每次执行前需司机实时确认（IM 弹窗）	超时未确认 → 自动取消

当前状态：L0-L2 生产运行 · L3 灰度中 · L4 设计完成待落地

工程原则

- 新能力默认从最严格层级开始，验证安全后逐级提升
 - 出问题随时降级——这是 **Agent** 能力扩展的"安全阀"
- 完整记录决策链路，出错可审计、可追溯

能力治理：评估驱动 × 能力演进

AGENTIC AI
超级智能体系统架构峰会



评估体系

模块正确性 – **WatchPlan** 对不对？**Fast Loop** 快不快？

WatchPlan合法率 编译稳定性 P99时延

行为合理性 – 推得不好、太多、还是太慢？

点击率 遗漏率 首次召回时延 曝N点1

业务有效性 – **Agent** 帮司机赚到钱了吗？

AI成交占比 推N成1率 留存率 匹配增量成交率

能力演进路径

看货（信息推送）

规则执行 · **Agent** 仅做意图理解



盯盘（持续执行）

规则 + **Agent** 协作 · 反馈闭环保证质量

← 推进中



经营规划（自主决策）

能力 + 信任双重门槛 · **Agent** 自主决策

对应关系

每开放一层新能力，先通过对应层级的评估验证安全性，确认后再放量。

模块正确 → 行为合理 → 业务有效，能力从看货→盯盘→经营规划逐步放开。

结语

三个工程实践

事件驱动 **AGENT LOOP**

Agent 不持续运行，
由业务事件唤醒、执行、休眠。

解决：成本 + 感知的确定性

意图编译为规则脚本

Agent 想明白一次，
规则引擎重复执行千万次。

解决：时延 + 成本 + 推货的确定性

动作分层授权

不同动作不同信任层级，
出错可审计、可回溯。

解决：信任 + 可控

让 **AGENT** 做判断，工程保证执行的确定性（可审计，可回溯）。

欢迎交流。

